

Group Project:

Multi-User Client-Server Messaging Application

Objective

To design and implement a **multi-user messaging system** based on a **client-server model** that demonstrates core **Operating System concepts** such as inter-process communication (IPC), synchronization, concurrency, socket programming, and persistent storage.

The project aims to give hands-on experience of building a '**real-world system**' that uses multiple OS concepts.

Components

The system will consist of two main components:

1. Server

- Runs as a process that handles connections from multiple clients (minimum of 5 concurrent users).
- Maintains user sessions, message queues, and communication between clients.
- Manages all message storage, retrieval, and deletion in a **server-side database** (can use SQLite or flat file storage).
- Performs appropriate cleanup and client notification during shutdown.

2. Client

- Runs as a standalone process that connects to the server using a simple command-line interface (CLI).
 - Provides user commands for sending, receiving, retrieving, and deleting messages.
 - Displays incoming messages in real time.
 - Disconnects gracefully upon shutdown or server termination.
-

Key Features

1. Connection Management

- Clients connect to the server using: `connect <server IP>`
- The server authenticates and registers each new client with a unique username.

- The server maintains a **list of active users**.

2. Messaging Functionality

- Send Message: *sendmessage <user> <message>*
Sends a message from the current user to the specified user.
- Receive Message
 - When a message arrives, the receiver's console displays:
 - Message from <sender>: <message text>
 - Messages are delivered in real-time if the user is online.

3. Message Storage and Retrieval

- *getmessages <user>*
 - Retrieves all previous messages exchanged with <user> from the server database.
- *deletemessages <user>*
 - Deletes all messages to/from <user> from the server-side database.

4. User List

- *getuserlist*
 - Displays a list of all currently connected users.

5. Shutdown Handling

- **Server Shutdown**
 - When the server shuts down, it sends a broadcast message:
"Server shutting down..."
 - All connected clients display this message and terminate gracefully.
- **Client Shutdown**
 - When a client exits, the server removes that user from the active user list.

Implementation Notes:

- Only server stores the messages
 - Clients (users) do not store anything locally. For users all chats are temporary. When a session ends, they don't retain anything.
 - Client can be a simple console app or a UI, whichever you prefer
 - Clients do not directly connect to other users. All messages are routed through server
 - A server could be a console application so that it shows some real-time messages of events like 'user ,x. connected' etc..
 - Server is obviously multi-threaded but client will also need threading
 - Ensure that client is able to connect to the server. Your endpoint security / firewall might prevent that. In such case, test by disabling security.
-

OS Concepts to Use

This project integrates several **core OS concepts** covered in our class:

Concept	Application in Project
Process Management	Each client and server instance runs as a separate process; the server handles multiple client processes concurrently.
Concurrency & Threading	The server uses threads to manage multiple simultaneous client connections.
Synchronization	Mutexes/semaphores used to manage shared data structures (user list, message queues, database access).
Inter-Process Communication (IPC)	Client-server communication implemented using sockets
I/O Management	Non-blocking I/O or select-based multiplexing for concurrent client handling.
File System & Persistence	Message storage implemented using a persistent database or flat files.
Signal Handling	Graceful shutdown using signals or a broadcast message

Deliverables

1. Design Document

Each group must submit a detailed **design document** that includes:

- System architecture diagram (client-server interaction, threads, and data flow).
- Design details of server and client – includes concurrency design of client server
- **Concurrency design** (how threads synchronize access to shared data).
- Testing strategy and unit tests

2. Source Code

- Fully functional **client** and **server** implementations in **C (Linux)**.
- Properly modularized and documented code.
- Clearly separated files for networking, threading, database, and command handling.

A single student from the group uploads all the deliverables as a ZIP file.

Rest of the students in the group just submit a dummy text file that has name of the student who submitted the assignment.

Code should be submitted one-day prior to the group presentations.

While presenting, first explain the design briefly, who has done what parts of the code and then a live demo.

1 student can run the server, others become clients.

Evaluation

Total Marks: 20

Criterion	Description	Weight
Functionality	Correct implementation of all required commands, concurrent clients, message handling, and persistence.	50%
Design & Documentation	Quality and clarity of design document, architectural decisions, and explanation of OS concepts used.	20%
Code and Tests	Code with meaningful comment and Unit Tests	15%
Demo	Ability to demonstrate, explain, and modify code on the spot. Each member should clearly understand the design and logic.	15%

Project Timeline (3 Weeks)

Demonstration Guidelines

During the evaluation:

You will download your own submission from moodle and demo that. No local code will be allowed.

You are expected to test this before the group presentation.

1. Run the server.
 2. Connect at least 5 clients concurrently.
 3. Demonstrate:
 - Sending and receiving messages between multiple users.
 - Retrieving and deleting message history.
 - Server shutdown broadcast.
 - Proper handling of concurrent operations.
 - Run Test cases
 4. Be ready to answer questions about thread handling, synchronization, and data structures.
-

All the Best!